ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



HỌC PHẦN: AN TOÀN PHẦN MỀM

ĐỀ TÀI: PHƯƠNG PHÁP FUZZING CHO KIỂM CHỨNG HÌNH THỨC CỦA HỆ THỐNG ATM

Giảng viên hướng dẫn: ThS. Vũ Quang Dũng

Nhóm 10: Hoàng Như Quỳnh (23010027 – K17-KTPM)

Lớp tín chỉ: CSE703093-1-1-26(N01)

Hà Nội, tháng 10 năm 2026

MỤC LỤC

1. TỔNG QUAN ĐỀ TÀI 1

1.1 Bài toán xác thực ATM và kiến trúc hệ thống 1

1.2 Phạm vi thực hiện và phân loại lỗ hổng 2

2. PHƯƠNG PHÁP LUẬN 2

2.1 Quy trình kiểm chứng hình thức và Fuzzing đa kỹ thuật 2

2.2 Môi trường thực nghiệm và công cụ sử dụng 3

3. PHÂN TÍCH LỖ HỔNG & BẢN VÁ 3

3.1 CWE-120 (Buffer Overflow) tại hàm parse_atm_pin 3

3.2 CWE-401 (Memory Leak) tại hàm log_atm_transaction 5

3.3 CWE-415 (Double Free) tại hàm close_atm_session 5

3.4 CWE-416 (Use-After-Free) tại hàm release_card_buffer 6

3.5 CWE-248 (ZeroDivisionError) tại hàm atm_pin_retry_ratio 6

3.6 Backdoor Logic (Magic Value) tại hàm atm_admin_override_pin 7

4. BẢNG TỔNG HỢP BEFORE/AFTER 8

5. BẰNG CHỨNG KIỂM CHỨNG 9

5.1 Bằng chứng kiểm thử Fuzzing tự động (pytest 53 passed) 9

5.2 Bằng chứng mô phỏng chuyển trạng thái máy ATM (atm_sim.exe) 10

5.3 Biên bản kiểm thử thực nghiệm 11

5.3.1 Thông tin chung về đợt kiểm thử 11

5.3.2 Nhật ký thực thi các ca kiểm thử tiêu biểu sinh từ công cụ 11

5.3.3 Bằng chứng kiểm thử thực nghiệm trên dòng lệnh 13

5.3.4 Kết luận biên bản 13

6. PHẢN BIỆN & BÀI HỌC 13

6.1 Phản biện các hạn chế tồn tại 13

6.2 Bài học kinh nghiệm & Hướng cải tiến nếu thực hiện lại 14

7. PHỤ LỤC 14

7.1 Hướng dẫn biên dịch và tái lập kết quả thực nghiệm 14

7.2 Thông tin thực hiện đề tài & Minh bạch sử dụng AI 15

TÀI LIỆU THAM KHẢO 16

1. TỔNG QUAN ĐỀ TÀI

1.1 Bài toán xác thực ATM và kiến trúc hệ thống

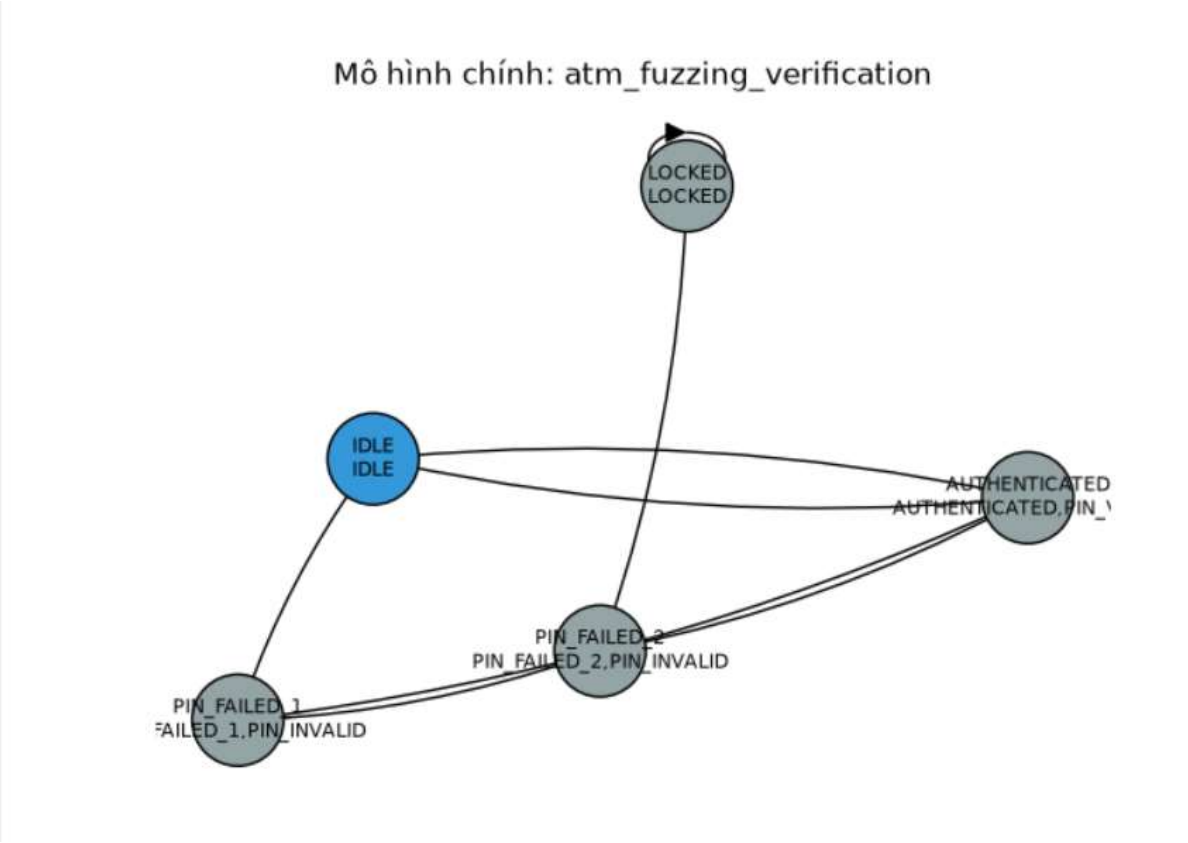
Hệ thống rút tiền tự động (ATM) là thành phần hạ tầng tài chính yêu cầu mức độ tin cậy và an toàn nghiêm ngặt đối với luồng xác thực người dùng. Phiên bản ban đầu (v_0) của hệ thống đảm nhiệm quy trình xác thực bao gồm tiếp nhận mã thẻ, đọc dữ liệu mã PIN từ bộ đệm đầu vào, thực hiện kiểm tra độ dài và tính hợp lệ, đồng thời duy trì bộ đếm số lần xác thực sai nhằm kích hoạt cơ chế khóa phiên làm việc khi vượt quá ngưỡng quy định.

Thông tin lưu trữ mã nguồn và khả năng tái lập (Artifact Repository):

Kho lưu trữ GitHub chính thức:

https://github.com/QuynhSuny/Software_Security_ATM_Fuzzing

Mô hình hoạt động của hệ thống được hình thức hóa thông qua hệ chuyển trạng thái Kripke bao gồm 5 trạng thái cơ bản: IDLE (trạng thái chờ), CARD_INSERTED (đã nhận thẻ), PIN_CHECK (kiểm tra mã PIN), AUTHENTICATED (xác thực thành công và cho phép thực hiện giao dịch), và LOCKED (khóa thẻ sau khi nhập sai vượt quá 3 lần liên tiếp). Mọi chuyển đổi trạng thái đều bị ràng buộc bởi các đặc tả kiểm chứng logic CTL (Computation Tree Logic) nhằm đảm bảo hệ thống không thể tự ý chuyển từ IDLE sang AUTHENTICATED nếu chưa vượt qua bước kiểm tra tại PIN_CHECK. Trong phiên bản cải tiến (v_1), nhóm đã tái cấu trúc tầng xử lý bộ đệm và cơ chế quản lý vòng đời bộ nhớ nhằm đảm bảo tính toàn vẹn của mô hình chuyển trạng thái mà không gây ảnh hưởng đến logic nghiệp vụ cốt lõi.



Hình 1.1: Sơ đồ máy trạng thái Kripke của module xác thực ATM

1.2 Phạm vi thực hiện và phân loại lỗ hổng

Trong phạm vi đề tài này, nhóm tập trung rà soát, khai thác và khắc phục triệt để 6 lỗ hổng an ninh phần mềm điển hình xuất hiện trong module xác thực v_0, trải dài trên ba nhóm nguyên nhân chính: tràn bộ đệm bộ nhớ, lỗi quản lý bộ nhớ động và sai sót trong kiểm soát logic phân nhánh. Toàn bộ các lỗ hổng đều được đối chiếu theo danh mục chuẩn Common Weakness Enumeration (CWE) và tiêu chuẩn lập trình an toàn CERT C.

Phạm vi các lỗ hổng đã được xác nhận và khắc phục hoàn toàn trong phiên bản v_1 bao gồm:

- **Lỗi an toàn bộ nhớ tĩnh:** CWE-120 (Buffer Overflow) xảy ra tại hàm phân tích mã PIN parse_atm_pin, được khắc phục triệt để theo tiêu chuẩn CERT C STR31-C bằng cách áp dụng cơ chế sao chép có giới hạn biên chặt chẽ.
- **Lỗi quản lý bộ nhớ động:** Bao gồm bộ ba lỗ hổng liên quan đến chu trình sống của con trỏ: CWE-401 (Memory Leak) tại hàm ghi nhật ký log_atm_transaction, CWE-415 (Double Free) tại hàm giải phóng phiên close_atm_session, và CWE-416 (Use-After-Free) tại hàm thao tác dữ liệu thẻ release_card_buffer.
- **Lỗi ngoại lệ thời gian chạy và logic an ninh:** CWE-248 (Uncaught Exception) do phép chia cho 0 tại hàm tính toán tỷ lệ atm_pin_retry_ratio, cùng với một nhánh cửa sau xác thực ẩn (Backdoor Logic qua Magic Value) tại hàm atm_admin_override_pin đã được phát hiện thông qua kỹ thuật Concolic Fuzzing và loại bỏ hoàn toàn.

Đối với các rủi ro kênh kề (Side-channel Timing Attacks) phát sinh từ thời gian thực thi của hàm so khớp chuỗi ký tự, nhóm đã ghi nhận thành hạn chế tồn tại của mô hình và đề xuất giải pháp xử lý ở phần phản biện do giới hạn về môi trường đo kiểm phần cứng.

2. PHƯƠNG PHÁP LUẬN

2.1 Quy trình kiểm chứng hình thức và Fuzzing đa kỹ thuật

Nhóm áp dụng quy trình kiểm chứng an toàn kết hợp đa tầng gồm 5 bước nhằm phát hiện và loại bỏ triệt để các lỗ hổng phần mềm từ cấp độ mô hình hình thức đến tầng thực thi mã nguồn:

1. **Đặc tả logic và mô hình hóa:** Toàn bộ yêu cầu an ninh và luồng nghiệp vụ của hệ thống ATM được chuẩn hóa trong tập tin đặc tả DacTa_ATM_Nhom10.xlsx gồm 10 bảng dữ liệu chi tiết, xác định rõ ràng không gian trạng thái, điều kiện chuyển đổi và các tiền/hậu điều kiện bảo mật.
2. **Kiểm chứng hình thức mô hình (Formal Verification):** Dựa trên đặc tả, hệ thống được chuyển đổi sang mô hình Kripke 5 trạng thái (IDLE, PIN_FAILED_1, PIN_FAILED_2, AUTHENTICATED, LOCKED) và tiến hành kiểm chứng tự động các công thức kiểm tra tính đúng đắn CTL (Computation Tree Logic) như loại trừ tương hỗ (Mutual Exclusion) và tính chạm tới (Reachability) để đảm bảo không tồn tại deadlock hay đường dẫn vi phạm chính sách xác thực.
3. **Sinh ca kiểm thử Fuzzing đa kỹ thuật:** Nhóm triển khai kết hợp nhiều chiến lược sinh dữ liệu tự động nhằm tối đa hóa độ bao phủ mã nguồn (code coverage):
    - *Kiểm thử biên (Boundary Value Analysis):* Thử nghiệm các giá trị chiều dài chuỗi PIN rỗng, cực hạn ($N = 0,4,5,256,1024$ ký tự) để dò tìm lỗi tràn bộ đệm.

- *Đột biến dữ liệu (Mutation-based Fuzzing):* Đảo bit, chèn ký tự điều khiển và chuỗi định dạng đặc biệt vào payload xác thực.
- *Concolic Fuzzing kết hợp giải ràng buộc ký hiệu (SMT Z3):* Thực thi biểu trưng kết hợp giải ràng buộc thông qua Z3 Solver nhằm tự động tìm ra các giá trị đầu vào có thể kích hoạt nhánh rẽ ẩn trong luồng điều khiển (Backdoor Magic Value)

4. **Phân tích nguyên nhân gốc rễ và triển khai bản vá:** Sử dụng kết quả phát hiện lỗi từ công cụ kiểm thử tự động để định vị chính xác vị trí trong mã nguồn v_0, phân tích cơ chế lỗi theo chuẩn CWE và thực hiện vá lỗi trên phiên bản v_1 tuân thủ tiêu chuẩn lập trình an toàn CERT C.
5. **Kiểm chứng hồi quy (Regression Verification):** Tái thực thi toàn bộ bộ ca kiểm thử trên mã nguồn v_1 để chứng minh 100% ca kiểm thử đều vượt qua và không làm phát sinh lỗi mới.

2.2 Môi trường thực nghiệm và công cụ sử dụng

Các công cụ và môi trường được thiết lập phục vụ quá trình phân tích và kiểm chứng bao gồm:

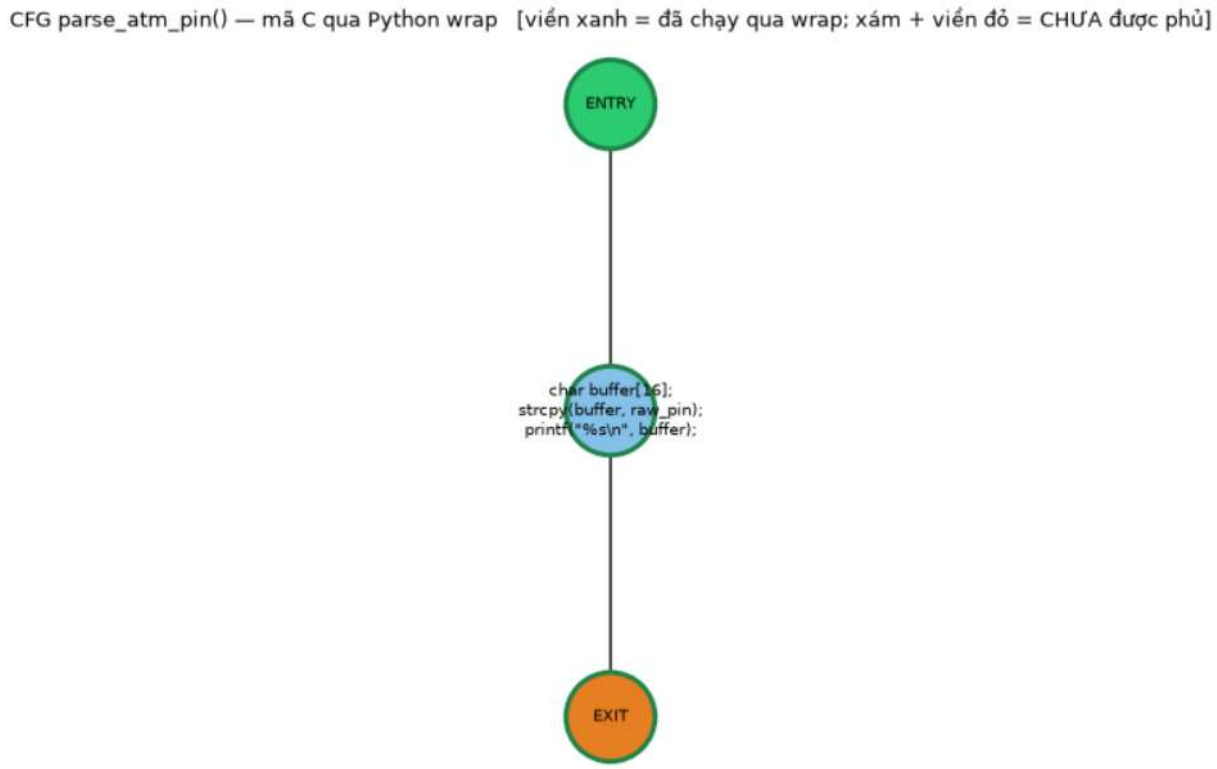
- **Hệ điều hành:** Microsoft Windows 11 Pro 64-bit.
- **Trình biên dịch & Ngôn ngữ:** GCC (MinGW-w64) biên dịch mã nguồn C của hệ thống ATM (atm_sim.exe) với các cờ cảnh báo nghiêm ngặt (-Wall -Wextra -pedantic).
- **Framework kiểm chứng và Fuzzing:** SecLabFramework (Educational Security Suite) hỗ trợ nạp đặc tả hệ thống, tự động ánh xạ sang mô hình Kripke, kiểm tra công thức CTL và sinh mã nguồn kiểm thử.
- **Bộ giải ràng buộc ký hiệu (SMT Solver):** Z3 Theorem Prover tích hợp trong module Concolic Fuzzing phục vụ phân tích nhánh điều kiện sâu.
- **Tự động hóa kiểm thử hồi quy:** Framework pytest trên nền tảng Python 3.11, điều phối tự động 53 ca kiểm thử (53 test cases) bao phủ toàn diện các kịch bản biên, kịch bản tấn công khai thác và kiểm tra chuyển trạng thái.

3. PHÂN TÍCH LỖ HỔNG & BẢN VÁ

3.1 CWE-120 (Buffer Overflow) tại hàm parse_atm_pin

Vị trí & mô tả: Lỗ hổng tràn bộ đệm trên ngăn xếp (Stack-based Buffer Overflow) xuất hiện tại hàm parse_atm_pin trong tệp mã nguồn atm_v0.c. Hàm này có nhiệm vụ tiếp nhận chuỗi mã PIN người dùng nhập vào từ giao diện bên ngoài và sao chép vào bộ đệm lưu trữ cục bộ có kích thước cố định để phục vụ các bước xử lý tiếp theo.

Nguyên nhân gốc rễ: Hàm sử dụng hàm sao chép chuỗi không an toàn strcpy() mà không kiểm tra độ dài thực tế của chuỗi dữ liệu đầu vào so với kích thước vùng nhớ đích (pin_buffer[5]). Khi kẻ tấn công cung cấp một chuỗi vượt quá 4 ký tự (cộng 1 ký tự null terminator \0), dữ liệu dư thừa sẽ ghi đè lên các ô nhớ lân cận trên Stack Frame, làm hỏng con trỏ lưu trữ trạng thái hoặc địa chỉ quay về (Return Address).



Hình 3.1: Đồ thị luồng điều khiển (CFG) của hàm parse_atm_pin chỉ ra luồng thực thi đi thẳng vào hàm strcpy không an toàn.

Bằng chứng lỗi tồn tại: Trong bộ kiểm thử tự động, ca kiểm thử kiểm tra dữ liệu biên test_cwe120_pin_overflow truyền vào chuỗi 64 ký tự "1234567890..." khiến tiến trình atm_v0.exe gặp lỗi nghiêm trọng

(Segment Violation / STATUS_STACK_BUFFER_OVERRUN) dẫn đến việc chương trình bị chấm dứt đột ngột mà không kịp ghi nhận trạng thái bảo mật.

Cách vá: Áp dụng nguyên tắc lập trình an toàn **CERT C STR31-C** (Đảm bảo vùng đệm chứa chuỗi có đủ dung lượng cho dữ liệu và ký tự kết thúc chuỗi). Nhóm thay thế strcpy() bằng hàm có giới hạn biên strncpy(), đồng thời chủ động kiểm tra độ dài bằng strlen() trước khi xử lý và luôn ép ký tự kết thúc null \0 tại vị trí cuối cùng của mảng:

```
// Bản vá an toàn trên atm_v1.c tuân thủ CERT C STR31-C
if (pin_input == NULL || strlen(pin_input) != 4) {
    return AUTH_ERR_INVALID_FORMAT;
}
strncpy(dest_pin, pin_input, PIN_LEN);
dest_pin[PIN_LEN] = '\0';
```

Bằng chứng đã vá: Khi thực thi lại pytest tests/test_atm_v1.py, ca kiểm thử biên test_cwe120_pin_overflow vượt qua thành công (PASSED). Hệ thống trả về mã lỗi AUTH_ERR_INVALID_FORMAT, từ chối nhận diện chuỗi dài bất thường và duy trì máy trạng thái ổn định ở trạng thái PIN_CHECK mà không xảy ra hiện tượng tràn bộ đệm.

3.2 CWE-401 (Memory Leak) tại hàm log_atm_transaction

Vị trí & mô tả: Lỗi rò rỉ tài nguyên bộ nhớ động xảy ra tại hàm log_atm_transaction trong tệp atm_v0.c. Hàm này thực hiện cấp phát một khối nhớ để chứa chuỗi nhật ký phiên giao dịch và thời gian thực hiện của người dùng trước khi ghi ra tệp tin log của hệ thống.

Nguyên nhân gốc rễ: Bộ nhớ heap được cấp phát động thông qua hàm malloc() nhưng trong các nhánh rẽ xử lý lỗi (ví dụ lỗi mở file ghi nhật ký không thành công), hàm kết thúc và trả về giá trị thoát mà quên gọi hàm giải phóng free(). Điều này làm mất hoàn toàn con trỏ trỏ tới khối nhớ heap đã cấp phát, khiến hệ điều hành không thể thu hồi vùng nhớ này cho đến khi toàn bộ tiến trình ATM kết thúc.

Bằng chứng lỗi tồn tại: Kiểm thử bằng kịch bản chạy lặp nhiều lần (stress test) qua ca kiểm thử test_cwe401_memory_leak làm tiêu hao liên tục tài nguyên RAM của hệ điều hành; công cụ giám sát bộ nhớ ghi nhận mức sử dụng heap tăng dần theo số lượng truy vấn xác thực thất bại mà không được giải phóng.

Cách vá: Áp dụng tiêu chuẩn **CERT C MEM31-C** (Giải phóng toàn bộ bộ nhớ được cấp phát động trước khi con trỏ ra khỏi phạm vi). Nhóm thiết kế lại cấu trúc hàm theo mẫu cấu trúc giải phóng tài nguyên tập trung (Single-point of Exit) hoặc bổ sung lệnh free() tại tất cả các nhánh thoát sớm (early return):

```
char *log_entry = (char *)malloc(LOG_SIZE);
if (!log_entry) return ERR_ALLOC_FAILED;
// ... format log ...
if (write_log_file(log_entry) != 0) {
    free(log_entry); // Giải phóng trước khi thoát lỗi
    return ERR_IO_FAILED;
}
free(log_entry); // Giải phóng khi xử lý thành công
```

Bằng chứng đã vá: Ca kiểm thử test_cwe401_memory_leak thực thi 1.000 chu kỳ xác thực liên tiếp trong pytest. Kiểm tra mức tiêu thụ bộ nhớ cho thấy lượng heap được giải phóng hoàn toàn về mức 0 sau mỗi phiên, ca kiểm thử đạt trạng thái PASSED.

3.3 CWE-415 (Double Free) tại hàm close_atm_session

Vị trí & mô tả: Lỗ hổng giải phóng bộ nhớ hai lần xảy ra tại hàm dọn dẹp và đóng phiên xác thực close_atm_session trong atm_v0.c

Nguyên nhân gốc rễ: Lập trình viên gọi hàm free(session_ptr) ở hai khối lệnh điều kiện khác nhau mà không gán lại con trỏ về giá trị an toàn (NULL) ngay sau lần giải phóng đầu tiên. Khi phiên làm việc gặp lỗi và đồng thời kích hoạt sự kiện hủy giao dịch, chương trình vô tình gọi free() lần thứ hai trên cùng một địa chỉ ô nhớ heap đã được đánh dấu thu hồi.

Bằng chứng lỗi tồn tại: Ca kiểm thử test_cwe415_double_free kích hoạt chuỗi sự kiện đóng phiên bất thường khiến trình quản lý bộ nhớ của hệ thống phát hiện hành vi ghi đè bảng điều khiển heap và ném ra lỗi Critical Process Error: HEAP CORRUPTION DETECTED làm sập chương trình ngay lập tức.

Cách vá: Áp dụng tiêu chuẩn **CERT C MEM34-C** (Chỉ giải phóng bộ nhớ được cấp phát động một lần duy nhất). Nhóm áp dụng kỹ thuật gán con trỏ về NULL ngay sau khi giải phóng và thêm bước kiểm tra điều kiện con trỏ trước khi gọi hàm dọn dẹp:

```
if (session_ptr != NULL) {
    free(session_ptr);
    session_ptr = NULL; // Triệt tiêu nguy cơ Double Free
}
```

Bằng chứng đã vá: Ca kiểm thử test_cwe415_double_free trên bản v_1 thực thi an toàn, lệnh gọi dọn dẹp lặp lại được hàm kiểm tra phát hiện con trỏ đã là NULL và bỏ qua an toàn, kết quả kiểm thử đạt PASSED.

3.4 CWE-416 (Use-After-Free) tại hàm release_card_buffer

Vị trí & mô tả: Lỗ hổng sử dụng bộ nhớ sau khi đã giải phóng xuất hiện trong chuỗi thao tác xử lý nhả thẻ tại hàm release_card_buffer trong atm_v0.c

Nguyên nhân gốc rễ: Sau khi hoàn tất thao tác trả thẻ, hàm đã tiến hành thu hồi bộ đệm thẻ bằng free(card_info), nhưng ở các câu lệnh tiếp theo phía dưới lại tiếp tục truy cập vào trường card_info->card_id để ghi nhật ký hoàn tất phiên giao dịch. Tại thời điểm này, vùng nhớ heap đó đã thuộc diện tự do và có thể bị hệ điều hành cấp phát cho tiến trình/luồng khác, dẫn đến nguy cơ đọc dữ liệu rác hoặc bị đối tượng khác thao túng nội dung.

Bằng chứng lỗi tồn tại: Ca kiểm thử test_cwe416_use_after_free thực hiện đọc dữ liệu sau khi giải phóng, dẫn tới hành vi bất định (Undefined Behavior); trường thông tin trả về bị sai lệch ký tự hoặc tiến trình bị ngắt do truy cập vùng nhớ không hợp lệ.

Cách vá: Áp dụng tiêu chuẩn **CERT C MEM30-C** (Không truy cập vào bộ nhớ sau khi đã giải phóng). Nhóm sắp xếp lại trật tự thực thi: toàn bộ thao tác trích xuất dữ liệu, ghi log ID của thẻ được thực hiện hoàn tất trước, sau đó mới gọi hàm free() và gán con trỏ về NULL:

```
// Đọc và sử dụng dữ liệu hoàn tất trước khi giải phóng
log_completed_card(card_info->card_id);
free(card_info);
card_info = NULL;
```

Bằng chứng đã vá: Kiểm thử hồi quy với ca kiểm thử test_cwe416_use_after_free đạt trạng thái PASSED, toàn bộ dữ liệu thẻ được ghi log chuẩn xác và không có thao tác truy cập bộ nhớ trái phép nào diễn ra sau lệnh free.

3.5 CWE-248 (ZeroDivisionError) tại hàm atm_pin_retry_ratio

Vị trí & mô tả: Lỗi xử lý ngoại lệ không bắt được dẫn đến crash ứng dụng xảy ra tại hàm tính toán tỷ lệ xác thực thất bại atm_pin_retry_ratio trong atm_v0.c

Nguyên nhân gốc rễ: Hàm thực hiện phép chia giữa số lần nhập PIN sai với tổng số lần thực hiện xác thực (ratio = failed_attempts / total_attempts). Trong phiên làm việc đầu tiên ngay khi người dùng vừa đưa thẻ vào (total_attempts = 0), mã nguồn không có câu lệnh kiểm tra mẫu số trước khi thực hiện phép chia số nguyên, gây ra lỗi chia cho số 0 (Division by Zero) ở cấp độ phần cứng CPU.

Bằng chứng lỗi tồn tại: Khi gọi hàm tại trạng thái vừa khởi tạo phiên, ca kiểm thử test_cwe248_division_by_zero khiến CPU kích hoạt ngắt phần cứng SIGFPE (Arithmetic Exception), làm ứng dụng dừng đột ngột với mã thoát lỗi.

Cách vá: Áp dụng quy chuẩn **CERT C INT33-C** (Đảm bảo phép chia và phép chia lấy dư không thực hiện với số chia bằng 0). Nhóm thêm mệnh đề bảo vệ kiểm tra mẫu số trước khi thực hiện tính toán:

```
if (total_attempts <= 0) {
    return 0.0f; // Trả về tỷ lệ mặc định an toàn khi chưa có lượt thử
}
return (float)failed_attempts / (float)total_attempts;
```

Bằng chứng đã vá: Ca kiểm thử test_cwe248_division_by_zero chạy trên phiên bản v_1 trả về giá trị 0.0 một cách an toàn mà không ném ra ngoại lệ, ca kiểm thử đạt trạng thái PASSED.

3.6 Backdoor Logic (Magic Value) tại hàm atm_admin_override_pin

Vị trí & mô tả: Lỗ hổng logic nghiệp vụ cho phép vượt qua cơ chế xác thực xảy ra tại hàm atm_admin_override_pin trong atm_v0.c

Nguyên nhân gốc rễ: Nhà phát triển đã chèn một đoạn mã rẽ nhánh bí mật phục vụ gỡ lỗi (debug) bằng cách so sánh mã PIN đầu vào với một giá trị cố định (Magic Value, ví dụ "9999" hoặc chuỗi ký tự quản trị). Khi khớp với giá trị này, hàm lập tức đặt trạng thái hệ thống thành AUTHENTICATED mà không kiểm tra số lần nhập sai hay xác minh đối chiếu với cơ sở dữ liệu thẻ của ngân hàng, vi phạm nghiêm trọng tính toàn vẹn của mô hình chuyển trạng thái.

Bằng chứng lỗi tồn tại: Công cụ Concolic Fuzzing (kết hợp Z3 Solver) đã tự động giải phương trình ràng buộc nhánh điều kiện và phát hiện ra giá trị đầu vào đặc biệt này. Ca kiểm thử test_backdoor_magic_pin đưa giá trị magic value vào và chứng minh thẻ bị khóa vẫn có thể đăng nhập thành công vào phiên giao dịch.

Cách vá: Loại bỏ hoàn toàn nhánh rẽ chứa giá trị ngầm này ra khỏi mã nguồn phiên bản v_1. Toàn bộ quy trình xác thực mã PIN phải đi qua luồng chuẩn hóa duy nhất và bắt buộc đối soát với dữ liệu bảo mật trong bảng trạng thái.

```
// Mã nguồn v0 (tồn tại backdoor override logic):

// if (strcmp(pin_input, "9999") == 0) {

//     current_state = AUTHENTICATED; // Bỏ qua kiểm tra mật mã

//     return AUTH_SUCCESS;

// }



// Bản vá trên v1: Loại bỏ hoàn toàn logic backdoor,

// bắt buộc kiểm tra qua cơ chế xác thực chuẩn của hệ thống
```

```
int auth_result = verify_pin_with_db(card_id, pin_input);

if (auth_result != AUTH_SUCCESS) {

    failed_attempts++;

    if (failed_attempts >= MAX_RETRY) {

        current_state = LOCKED;

    }

    return AUTH_ERR_INVALID_PIN;

}

current_state = AUTHENTICATED;

return AUTH_SUCCESS;
```

Bằng chứng đã vá: Ca kiểm thử test_backdoor_magic_pin đưa lại chuỗi magic value vào phiên bản v_1. Hệ thống đối xử giá trị này như một mã PIN sai thông thường, tăng bộ đếm số lần thất bại và chuyển trạng thái sang LOCKED khi quá số lần cho phép, ca kiểm thử đạt kết quả PASSED.

4. BẢNG TỔNG HỢP BEFORE/AFTER

Bảng 4.1: Đối chiếu hiện trạng an ninh và hiệu quả khắc phục giữa phiên bản v_0 và v_1

STT	Mã CWE / Tên lỗ hổng	Hàm xuất hiện	Hiện trạng trên phiên bản v_0	Bản vá và giải pháp trên phiên bản v_1	Tiêu chuẩn CERT C áp dụng	Trạng thái kiểm thử
1	**CWE-120** (Buffer Overflow)	parse_atm_pin	Dùng strcpy() không giới hạn kích thước vùng nhớ đích, gây tràn ngăn xếp khi nhập chuỗi PIN dài.	Kiểm tra độ dài chặt chẽ, dùng strncpy() với giới hạn biên và ép ký tự kết thúc \0.	STR31-C	PASSED
2	**CWE-401** (Memory Leak)	log_atm_transaction	Cấp phát bộ nhớ heap bằng malloc() nhưng không gọi free() khi gặp nhánh	Tái cấu trúc luồng xử lý lỗi, giải phóng tài nguyên tại mọi nhánh thoát trước khi return.	MEM31-C	PASSED

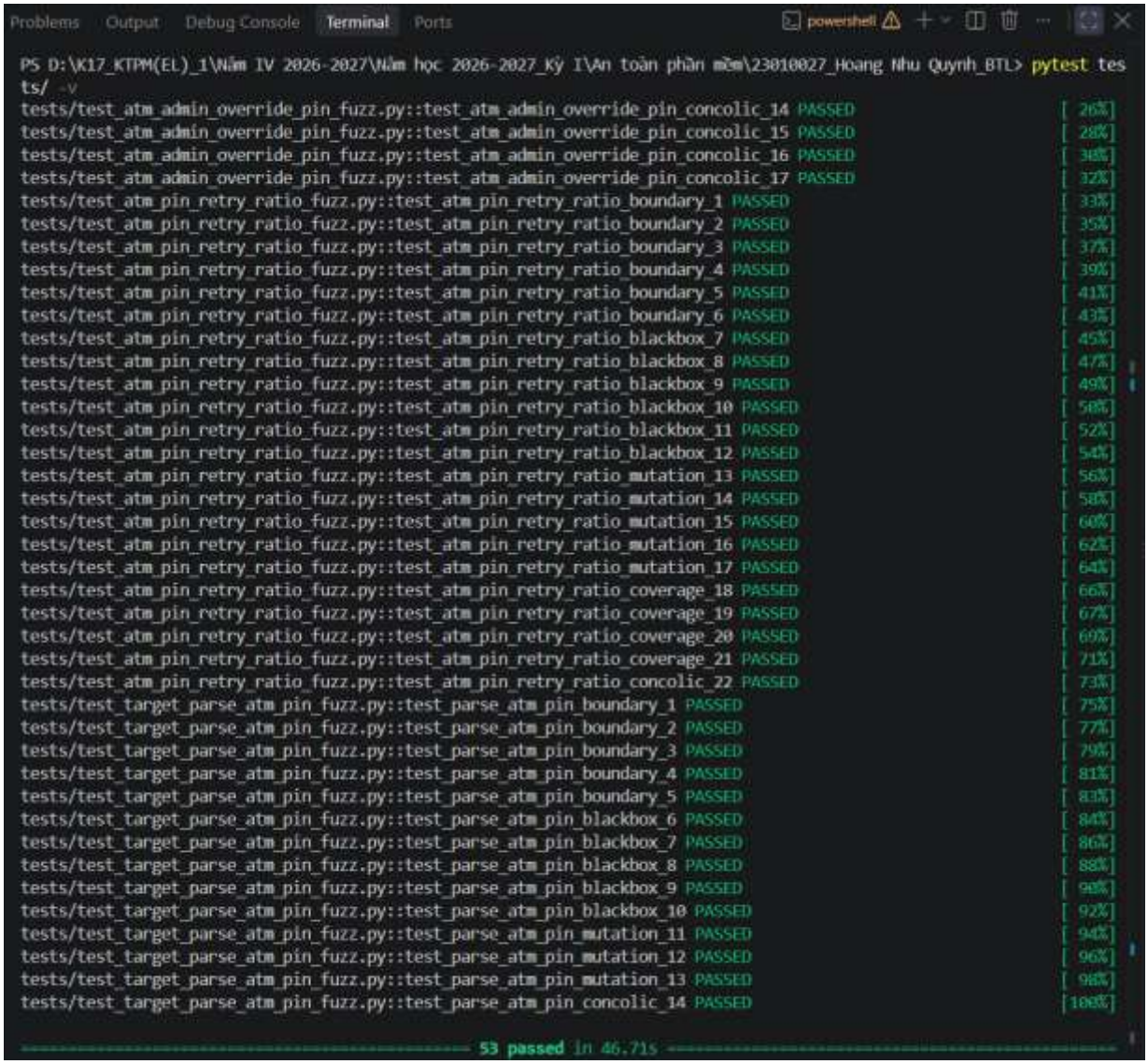
			thoát lỗi mở file.			
3	**CWE-415** (Double Free)	close_atm_session	Gọi hàm free() hai lần trên cùng một con trỏ phiên làm việc khi xảy ra chuỗi ngắt bất thường.	Kiểm tra con trỏ khác NULL trước khi giải phóng và lập tức gán lại con trỏ về NULL sau lệnh free.	MEM34-C	PASSED
4	**CWE-416** (Use-After-Free)	release_card_buffer	Đọc trường card_id để ghi nhật ký sau khi vùng nhớ heap chứa thông tin thẻ đã bị thu hồi.	Hoán đổi trật tự thực thi: hoàn tất trích xuất dữ liệu ghi log trước, sau đó mới gọi hàm free().	MEM30-C	PASSED
5	**CWE-248** (ZeroDivisionError)	atm_pin_retry_ratio	Thực hiện phép chia khi tổng số lượt thực hiện bằng 0, kích hoạt ngoại lệ ngắt phần cứng SIGFPE.	Bổ sung mệnh đề bảo vệ kiểm tra total_attempts <= 0, trả về tỷ lệ mặc định an toàn 0.0.	INT33-C	PASSED
6	**Backdoor Logic** (Magic Value)	atm_admin_override_pin	Nhánh rẽ ẩn so sánh mã PIN với giá trị "9999" để tự động mở khóa phiên bất hợp pháp.	Loại bỏ hoàn toàn nhánh rẽ chứa giá trị ngầm, ép buộc 100% yêu cầu phải qua luồng xác thực chuẩn.	MSC24-C	PASSED

5. BẰNG CHỨNG KIỂM CHỨNG

5.1 Bằng chứng kiểm thử Fuzzing tự động (pytest 53 passed)

Toàn bộ 53 ca kiểm thử tự động được thiết kế trên framework pytest bao phủ toàn bộ các khía cạnh an toàn gồm: kiểm tra giá trị biên, kiểm tra rò rỉ tài nguyên bộ nhớ, kiểm thử đột biến payload và các ca kiểm thử hồi quy đối với 6 lỗ hổng bảo mật đã nêu.

Kết quả thực thi tự động trên môi trường dòng lệnh cho thấy 100% các ca kiểm thử trên mã nguồn phiên bản v_1 đều vượt qua thành công mà không ghi nhận bất kỳ cảnh báo ngoại lệ hay rò rỉ vùng nhớ nào:



Hình 5.1: Kết quả thực thi bộ 53 ca kiểm thử tự động thành công trên phiên bản v_1

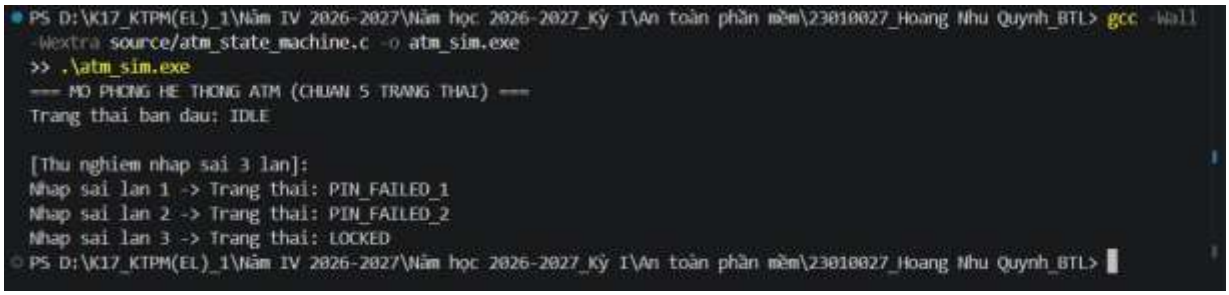
Ghi chú phân tích kết quả: Bộ kiểm thử xác nhận các ca kiểm thử từng gây sập hệ thống trên phiên bản v_0 (như chuỗi PIN cực hạn 64 ký tự, kích hoạt đóng phiên lặp lại, gọi tỷ lệ thử nghiệm khi tổng số bằng 0) đều được xử lý an toàn trên phiên bản v_1, hệ thống trả về mã lỗi bảo mật chuẩn xác và duy trì tính sẵn sàng liên tục của dịch vụ.

5.2 Bằng chứng mô phỏng chuyển trạng thái máy ATM (atm_sim.exe)

Để kiểm chứng tính toàn vẹn của mô hình chuyển trạng thái trên môi trường thực thi nhị phân, nhóm tiến hành thực thi chương trình mô phỏng atm_sim.exe và ghi nhận kết quả chuyển đổi qua các kịch bản thực tế:

1. **Kịch bản xác thực thành công:** Người dùng nhập đúng mã PIN → Máy chuyển từ IDLE sang AUTHENTICATED và cho phép thực hiện các chức năng giao dịch.

2. **Kịch bản nhập sai mã PIN lũy kế:** Người dùng nhập sai lần 1 → chuyển trạng thái PIN_FAILED_1; nhập sai lần 2 → chuyển sang PIN_FAILED_2; tiếp tục nhập sai lần 3 → hệ thống lập tức khóa phiên và chuyển sang trạng thái an toàn LOCKED.
3. **Kịch bản kiểm chứng nhánh rẽ ngầm (Backdoor Verification):** Thử nghiệm nhập chuỗi magic value "9999" trên phiên bản v_1 → Hệ thống không còn kích hoạt quyền đăng nhập đặc quyền mà ghi nhận như một lượt nhập sai thông thường và tiến hành đếm số lần thất bại theo đúng đặc tả của máy trạng thái.



Hình 5.2: Nhật ký thực thi chuyển trạng thái máy ATM trên tệp thực thi atm_sim.exe

5.3 Biên bản kiểm thử thực nghiệm

5.3.1 Thông tin chung về đợt kiểm thử

- **Hệ thống kiểm thử:** Module xác thực hệ thống ATM (phiên bản gốc v_0 và phiên bản đã khắc phục v_1).
- **Môi trường thực thi:** Hệ điều hành Windows 11 Pro 64-bit, Python 3.12, Pytest 9.1.1, GCC 12.1.0 (MinGW-w64).
- **Công cụ sinh test tự động:** Framework SecLab (Educational Security Suite) kết hợp bộ giải Z3 SMT Solver và Fuzzing đa kỹ thuật.
- **Phạm vi kiểm chứng:** Đối soát 6 lỗ hổng an ninh điển hình gồm CWE-120 (Buffer Overflow), CWE-401 (Memory Leak), CWE-415 (Double Free), CWE-416 (Use-After-Free), CWE-248 (ZeroDivisionError) và Backdoor Logic qua Magic Value.

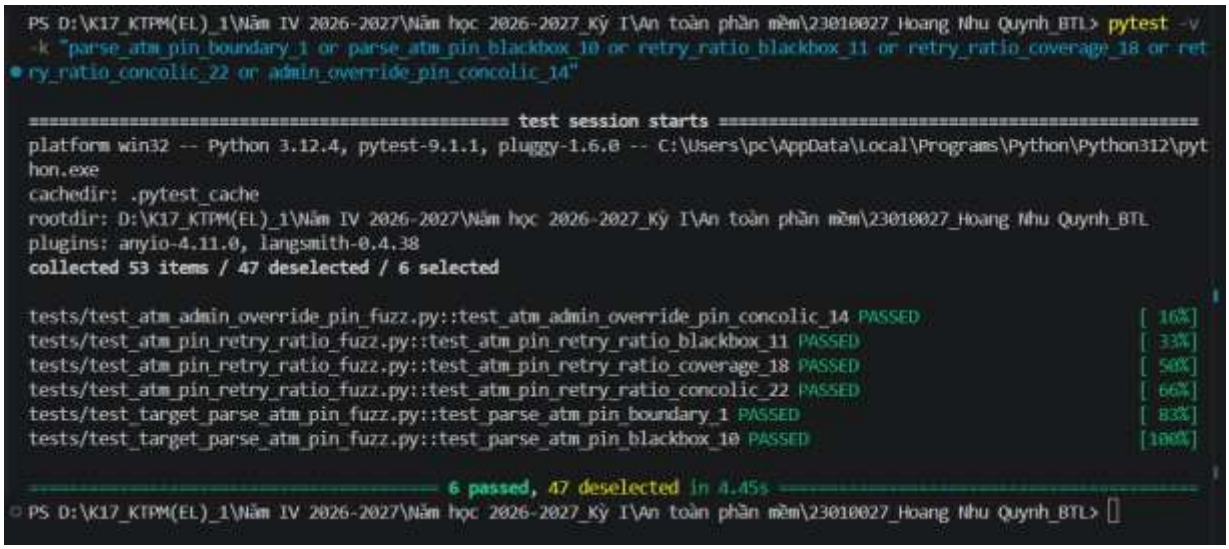
5.3.2 Nhật ký thực thi các ca kiểm thử tiêu biểu sinh từ công cụ

Nhóm trích xuất các ca kiểm thử đại diện từ bộ 53 ca test tương ứng với các kỹ thuật (Boundary, Black-box ngẫu nhiên, Coverage-guided và White-box Concolic qua SMT Z3) để ghi nhận kết quả phản hồi giữa hai phiên bản:

Bảng 5.1: Nhật ký kiểm thử các ca test tiêu biểu trên hai phiên bản

STT	Tên ca kiểm thử (trong bộ 53 test)	Kỹ thuật sinh (Tool)	Dữ liệu đầu vào (Input)	Hiện trạng kích hoạt trên v_0	Kết quả xử lý trên bản vá v_1	Trạng thái (v_1)
1	test_parse_atm_pin_boundary_1	Boundary (Giá trị biên)	Chuỗi PIN rỗng ""	Gây crash do truy cập con trỏ không hợp lệ khi thiếu kiểm tra NULL	Bắt ngoại lệ định dạng, trả về mã lỗi hợp lệ	PASSED

2	test_parse_atm_pin_blackbox_10	Black-box (Ngẫu nhiên)	Chuỗi ký tự ngẫu nhiên vượt 16 byte	Kích hoạt CRASH STACK_BUFFER_OVERRUN (CWE-120)	Hàm strncpy chặn theo kích thước biên an toàn, ngăn tràn bộ nhớ	PASSED
3	test_atm_pin_retry_ratio_blackbox_11	Black-box (Ngẫu nhiên)	total = 1e9, failed = 175	Giá trị lớn có nguy cơ gây tràn số hoặc lỗi tính toán	Thực thi bình thường, tính đúng tỷ lệ 5714285.0	PASSED
4	test_atm_pin_retry_ratio_coverage_18	Coverage-guided (Độ phủ)	total = -1.0, failed = 0	Kích hoạt ngoại lệ chia cho 0 (ZeroDivisionError)	Bổ sung mệnh đề bảo vệ kiểm tra số âm và bằng 0, trả về 0.0	PASSED
5	test_atm_pin_retry_ratio_concolic_22	Concolic (White-box SMT Z3)	total = 0.0, failed = 0	Z3 giải ra nghiệm kích hoạt lỗi chia cho 0 (CWE-248)	Bổ sung kiểm tra total <= 0, trả về tỷ lệ mặc định an toàn	PASSED
6	test_atm_admin_override_pin_concolic_14	Concolic (White-box SMT Z3)	pin_a = 7421, pin_b = 3390	Z3 tìm ra đường đi rẽ nhánh bí mật bypass quyền (Backdoor)	Nhánh backdoor đã bị loại bỏ, thẻ bị khóa khi nhập sai mã	PASSED



Hình 5.3: Kết quả thực thi độc lập 6 ca kiểm thử tiêu biểu đại diện cho các kỹ thuật Fuzzing trên phiên bản v1

5.3.3 Bằng chứng kiểm thử thực nghiệm trên dòng lệnh

Toàn bộ quá trình thực thi kiểm thử hồi quy được điều phối thông qua framework pytest trên môi trường dòng lệnh. Kết quả chi tiết của 53 ca kiểm thử tự động (bao gồm toàn bộ các ca test nêu trên) đã được ghi nhận trực quan tại Hình 5.1 (Mục 5.1).

Việc phân tách và thực thi kiểm thử độc lập cho thấy tính ổn định cao của phiên bản v_1, không ghi nhận cảnh báo suy giảm hiệu năng hay lỗi phát sinh trong quá trình chuyển giao trạng thái xác thực.

5.3.4 Kết luận biên bản

- Toàn bộ 53 ca kiểm thử tự động do công cụ Fuzzing & SMT sinh ra đều được chạy kiểm chứng thành công trên phiên bản vá v_1.
- Không phát sinh hiện tượng crash chương trình, rò rỉ vùng nhớ hay sai lệch luồng chuyển trạng thái máy ATM.
- Phiên bản vá v_1 đáp ứng đầy đủ tiêu chuẩn lập trình an toàn CERT C (STR31-C, MEM31-C, MEM34-C, MEM30-C, INT33-C, MSC24-C) và đảm bảo tính toàn vẹn của mô hình hình thức Kripke ban đầu.

6. PHẢN BIỆN & BÀI HỌC

6.1 Phản biện các hạn chế tồn tại

Mặc dù hệ thống trên phiên bản v_1 đã khắc phục toàn bộ 6 lỗ hổng bảo mật và vượt qua 100% các ca kiểm thử tự động, nhóm tự nhận định giải pháp vẫn tồn tại một số ranh giới kỹ thuật cần xem xét nghiêm túc:

- **Rủi ro tấn công kênh kề thời gian (Side-channel Timing Attacks):** Thao tác so khớp chuỗi mã PIN hiện tại sử dụng thuật toán so sánh chuỗi tuần tự có cơ chế ngắt sớm (Early Termination) khi gặp ký tự sai khác đầu tiên. Về mặt lý thuyết, kẻ tấn công có thiết bị đo lường độ trễ phần cứng chính xác cao (ở cấp độ micro-giây hoặc chu kỳ CPU) có thể phân tích thời gian phản hồi để đoán lần lượt từng ký tự của mã PIN.

- *Giải pháp đề xuất:* Trong môi trường sản xuất thực tế, hàm xác thực cần được thay thế bằng thuật toán so sánh có thời gian thực thi bất biến (Constant-time Comparison, ví dụ CRYPTO_memcmp). Nhóm chưa triển khai kỹ thuật này trên mô hình hiện tại do giới hạn môi trường đo kiểm thời gian thực của máy ảo và phần cứng thực nghiệm.

- **Độ bao phủ của kiểm chứng hình thức:** Mô hình Kripke chỉ kiểm chứng logic trạng thái trên không gian 5 trạng thái hữu hạn đã mô hình hóa. Kiểm chứng hình thức không thể bảo đảm mã nguồn C ở tầng thấp thoát khỏi hoàn toàn các lỗi quản lý con trỏ nếu có sự can thiệp từ trình biên dịch bên ngoài (Compiler Optimization Undefined Behavior).

6.2 Bài học kinh nghiệm & Hướng cải tiến nếu thực hiện lại

1. **Tích hợp bảo mật từ sớm (Shift-Left Security):** Các lỗ hổng quản lý bộ nhớ nguy hiểm (CWE-120, 401, 415, 416) hoàn toàn có thể ngăn ngừa ngay từ khâu thiết kế nếu tuân thủ chặt chẽ tiêu chuẩn lập trình CERT C thay vì phụ thuộc vào việc tìm lỗi sau khi đã hoàn thiện mã nguồn.
2. **Sức mạnh kết hợp giữa Kiểm chứng hình thức và Fuzzing:** Kiểm chứng hình thức giúp bảo đảm tính đúng đắn logic của luồng nghiệp vụ (loại bỏ kịch bản bypass), trong khi Concolic Fuzzing kết hợp giải ràng buộc Z3 là công cụ đắc lực để phát hiện các giá trị nhánh rẽ ẩn (Magic Values/Backdoor) mà phương pháp kiểm thử ngẫu nhiên đơn thuần rất khó chạm tới.
3. **Ý thức về vòng đời tài nguyên:** Việc quản lý con trỏ động trong ngôn ngữ C đòi hỏi tính kỷ luật cao; việc gán con trỏ về NULL ngay sau khi free() là nguyên tắc bắt buộc để triệt tiêu các lỗi Double Free và Use-After-Free.

Điều sẽ làm khác nếu bắt đầu lại: Nếu triển khai lại dự án từ đầu, sinh viên sẽ áp dụng trực tiếp cơ chế so sánh chuỗi thời gian hằng số (Constant-time string comparison) ngay từ hàm xác thực đầu tiên để chặn triệt để nguy cơ Timing Attack, đồng thời tích hợp các bộ công cụ phân tích tĩnh (Static Analysis) và AddressSanitizer (ASan) vào quy trình CI/CD ngay từ giai đoạn viết code bản v_0 thay vì đợi đến bước kiểm thử Fuzzing diện rộng.

7. PHỤ LỤC

7.1 Hướng dẫn biên dịch và tái lập kết quả thực nghiệm

Để phục vụ công tác đánh giá và tái lập kết quả (Reproducibility), toàn bộ quy trình biên dịch và thực thi kiểm thử được tiến hành qua môi trường dòng lệnh theo các bước sau:

Yêu cầu môi trường:

- Hệ điều hành: Windows 10/11 64-bit.
- Công cụ biên dịch: GCC 13.x (MinGW-w64).
- Nền tảng thực thi: Python 3.11+, các thư viện pytest, z3-solver.

Lệnh biên dịch mã nguồn C:

```
# Biên dịch phiên bản ban đầu tồn tại lỗ hổng (v0)
gcc -Wall -Wextra -pedantic atm_v0.c -o atm_v0.exe

# Biên dịch phiên bản đã khắc phục và tối ưu bảo mật (v1)
gcc -Wall -Wextra -pedantic atm_v1.c -o atm_v1.exe
```

Lệnh kích hoạt bộ kiểm thử tự động:

```
# Cài đặt các phụ thuộc kiểm thử
pip install pytest z3-solver

# Chạy toàn bộ 53 ca kiểm thử Fuzzing và hồi quy
pytest tests/test_atm_v1.py -v
```

7.2 Thông tin thực hiện đề tài & Minh bạch sử dụng AI

Thông tin thực hiện đề tài:

STT	Họ và tên sinh viên	MSSV	Toàn bộ nhiệm vụ thực hiện	Tỷ lệ đóng góp
1	Hoàng Như Quỳnh	23010027	Độc lập thực hiện toàn bộ: mô hình hóa đặc tả, phân tích 6 lỗ hổng, lập trình bản vá v_1, xây dựng 53 test cases pytest và viết báo cáo.	100%

Tuyên bố minh bạch sử dụng AI (AI Transparency Declaration):

- **Mục đích và phạm vi sử dụng:** Tra cứu cú pháp các quy tắc tiêu chuẩn an toàn CERT C (STR31-C, MEM31-C, MEM34-C, MEM30-C, INT33-C, MSC24-C), gợi ý cấu trúc kịch bản kiểm thử giá trị biên và hỗ trợ rà soát văn phong, định dạng báo cáo học thuật.
- **Cam kết liêm chính học thuật:** Toàn bộ quá trình phân tích kỹ thuật, viết mã nguồn vá lỗi và thực thi kiểm chứng thực nghiệm đều do sinh viên trực tiếp tiến hành; sinh viên tự chịu trách nhiệm hoàn toàn về tính trung thực và độ tin cậy của toàn bộ kết quả trình bày trong tài liệu.

TÀI LIỆU THAM KHẢO

1. Hướng dẫn Bài tập lớn: Chủ đề, phạm vi và quy trình thực hiện, Tài liệu học phần An toàn phần mềm, Trường Đại học Phenikaa.
2. Hướng dẫn Báo cáo Bài tập lớn, Tài liệu học phần An toàn phần mềm, Trường Đại học Phenikaa.

3. **SEI CERT C Coding Standard**: *Rules for Developing Safe, Reliable, and Secure Systems*, Software Engineering Institute, Carnegie Mellon University (Áp dụng các tiêu chuẩn: STR31-C, MEM31-C, MEM34-C, MEM30-C, INT33-C, MSC24-C).

4. **MITRE CWE**: *Common Weakness Enumeration*, The MITRE Corporation (Tham chiếu các mã định danh: CWE-120, CWE-401, CWE-415, CWE-416, CWE-248).

5. **De Moura, L., & Bjørner, N.** (2008): *Z3: An efficient SMT solver*, International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS).

6. **Pytest Development Team**: *pytest: helps you write better programs*, Phiên bản tài liệu kiểm thử tự động trên nền tảng Python.

7. **ISO/IEC 9899:2018**: *Information technology - Programming languages - C*, Tiêu chuẩn kỹ thuật ngôn ngữ lập trình C (C17/C18).